

METALOGOS

Phase 2 Close Report & Phase 3 Launch

AI-native programming language

Version 0.3.0 | 2026-06-01
GitHub: [ShkodnikAI/Metalogos-](#)

1. Phase 2 Close Summary

Phase 2 of the METALOGOS project is officially closed. This phase introduced knowledge graph capabilities, full adapt with sandbox and rollback, and semantic memory traversal. All four sub-phases (Phase 2 base, Phase 2.1 vector recall, Phase 2.3 ML backend, Phase 2 Final) are complete, tested, and pushed to the GitHub repository at ShkodnikAI/Metalogos-. The interpreter now supports graph-aware recall, mutation with accuracy-based rollback decisions, and sandbox declarations with allowed/forbidden/timeout constraints. The trait-based backend architecture provides clean extension points for future integration with real ML infrastructure.

1.1 Sub-phase Deliverables

Sub-phase	Feature	Key Deliverables	ADR
P2 base	Knowledge Graph, Full Adapt	relate, sandbox, mutate with rollback_if, graph-aware recall	0014, 0015
P2.1	Vector Recall	EmbeddingBackend trait, TF-IDF similarity, vector memory search	(merged)
P2.3	ML Backend	MLBackend trait, MockMLBackend, learn with data/epochs	0013
P2 Final	Graph + Adapt	Built-in graph, Relation edges, sandbox, rollback	0014, 0015

Table 1: Phase 2 sub-phase deliverables

The knowledge graph uses an in-memory graph structure with Relation edges linking memory entries. The recall builtin traverses the graph, returning the matched memory entry plus any related nodes connected through Relation edges. This enables graph-aware reasoning without external dependencies like Neo4j, keeping the interpreter self-contained. The full adapt system implements mutation with rollback: when a mutate declaration specifies `rollback_if` (e.g., accuracy less than 0.9), the interpreter computes a mock accuracy and rolls back if the condition is met. Sandbox declarations record allowed and forbidden operations with timeout constraints.

2. Phase 3: CLI, REPL, and Semantic Check

Phase 3 introduces developer tooling. The project now has three CLI subcommands, an interactive REPL with persistent state, and a semantic analysis mode that validates programs without executing them. This transforms METALOGOS from a single-command runner into a proper interactive development environment. The implementation uses clap for CLI argument parsing and rustyline for REPL line editing with history persistence.

2.1 CLI Subcommands

Command	Description	Implementation
mlog run <file>	Parse and execute a .mlog program	metalogos::run_program()
mlog repl	Interactive session with persistent state	rustyline 14, feed_line(), Interpreter reuse
mlog check <file>	Semantic analysis without execution	semantic::check_program(), two-pass validation

Table 2: CLI subcommands in Phase 3

2.2 REPL Architecture

The REPL operates in two modes. In interactive mode (stdin is a tty), rustyline provides line editing with history persisted to ~/.mlog_history, a "mlog> " prompt, and output prefixed with "=> ". In piped mode (stdin redirected, used by integration tests), lines are read silently from stdin, evaluated, and results printed. The METALOGOS_FORCE_PIPE=1 env var forces piped mode for test reproducibility. The key architectural element is feed_line() in lib.rs, a thin wrapper that parses a single line and feeds it to the existing Interpreter::run() method. The Interpreter instance persists across REPL iterations, so all state survives between inputs.

2.3 Semantic Check

The mlog check command performs two-pass static analysis without execution. Pass 1 collects all declared names into sets, detecting duplicates. Pass 2 cross-references all declarations: entity records reference valid types, rule targets reference existing entities, adapt/mutate targets reference existing learnable patterns, and flow pipeline steps reference known patterns or builtins. AnalysisResult holds separate errors and warnings

vectors; errors cause exit code 1, clean programs output "OK: no issues found."

3. Test Matrix

The complete test suite contains 11 tests across four test files, all passing with zero warnings. Golden tests auto-discover `.mlog/.expected` pairs in `examples/`. Semantic unit tests validate `check_program` analysis. Check integration tests exercise the full pipeline through the public API. The REPL integration test validates that three incremental `feed_line()` calls produce the correct output.

Test File	N	Covers
tests/golden.rs	1	8 golden pairs (m1-m5, p1_fluid, p2_graph, p2_adapt)
src/semantic.rs	4	ok_program, undefined_type, adapt_target, duplicate_pattern
tests/check_integration.rs	5	ok, undefined_type, adapt_target, duplicate_entity, format
tests/repl_integration.rs	1	Three incremental feed_line() calls with state persistence

Table 3: Complete test matrix (11 tests, all green)

4. Architecture Overview

The project follows the Metalogos Build Ladder contract-first TDD methodology. Each feature begins with a `.mlog` contract and `.expected` output file. The trait-based backend architecture (`LlmBackend`, `EmbeddingBackend`, `MlBackend`) allows seamless mock/real switching. `LlmBackend`, `EmbeddingBackend`, and `MlBackend` each have mock implementations for tests and real implementations gated behind environment variables.

File	Responsibility
src/main.rs	CLI entry: clap subcommands (run, repl, check), REPL loop
src/lib.rs	Public API: run_program(), check_program(), feed_line()
src/interpreter.rs	Tree-walking interpreter, 7 pillars, graph recall, mutate
src/semantic.rs	Two-pass semantic analysis, AnalysisResult, cross-refs

src/parser.rs	pest PEG parser, Rule::program to AST conversion
src/ast.rs	All AST types: Declaration, Expr, Statement, Condition
src/grammar.pest	PEG grammar for .mlog language
src/builtins.rs	Builtins: upper, lower, len, str, print, contains, float
src/llm.rs	LlmBackend trait, MockLlm, RealLlm (curl)

Table 4: Source file responsibilities

5. Architecture Decision Records

ADRs document all significant design decisions with context, decision, consequences, and prior art references. The following table lists the complete ADR index.

AD R	Phase	Topic
0001	M1	Architecture: pest, tree-walking interpreter, project structure
0002	M2	Rule engine, flow branching, pest lessons
0003	M3	Learnable patterns, LLM trait (MockLlm/RealLlm)
0004	M4	Memory: memorize/recall/forget with ACT-R decay
0005	M5	Adapt: few-shot cache, safety invariants
0006	P1	Fluid Types: lazy collapse, soft-failure
0013	P2.3	ML Backend: MlBackend trait, MockMlBackend
0014	P2	Knowledge Graph: relate, graph-aware recall
0015	P2	Full Adapt: sandbox, mutate, rollback
0016	P3	CLI + REPL + Semantic Check: clap, rustyline, feed_line

Table 5: ADR index

6. What Comes Next

Phase 3 developer tooling (CLI, REPL, check) is now complete. The remaining Phase 3 items from the Build Ladder roadmap include LSP integration for editor support, a package manager for sharing .mlog programs, a standard library, and documentation generation. Phase 4 targets bytecode-VM/JIT compilation, optimizations, and self-hosting the compiler in Metalogos itself. The trait-based backend architecture provides clear extension points for real ML infrastructure integration (PyO3 + PyTorch, sentence-transformers, OpenAI/Ollama LLM backends).

Phase	Planned Work	Status
Phase 3 (cont.)	LSP, package manager, stdlib, docs	Next orders
Phase 4	Bytecode-VM/JIT, optimizations, self-hosting	Roadmap
Backend upgrade	Real LLM, PyO3 + PyTorch, sentence-transformers	Feature gates

Table 6: Roadmap and next steps